

CODING AGENTS WITH AN OBSTACLE-AWARE HARNESS FOR SAFE ROBOT MANIPULATION

Bingxin Xu¹ Yuzhang Shang² Zhen Dong³ Emilio Ferrara¹
¹USC ²UCF ³UCSB

ABSTRACT

Coding agents have emerged as a promising paradigm for robot manipulation: a language model writes the robot controller as a program, and agents built in this way now operate robots without robot-specific training. Whether this paradigm is also safe, however, has not been asked. We evaluate coding agent under a safety constraint, where each task pairs a manipulation goal with an obstacle the robot must not touch. The agent pursues the goal but collides with the obstacle in most cases, treating task completion as its sole objective while neglecting safety. The agent reasons about the obstacle in its traces, and the prompt already forbids touching it, so neither perception nor instruction is at fault; the fault lies in the planning, where the stated constraint never becomes a priority. By decomposing manipulation into a route phase and a contact-rich moment, we locate the source of the failure. Along the route, the model cannot prioritize the safety constraint, having no notion of a clearing route and none of replanning once a chosen route becomes infeasible. At the contact, it is unaware that contact execution is bounded by the same constraint. To close this gap, we present **SAFEHARNESS**, which equips the model with two obstacle-aware harnesses that enable it to prioritize the safety constraint. Obstacle-aware route planning grounds the objects as bounding boxes and draws candidate routes over them as sequences of waypoints. The agent then plans a route in advance, verifies it, replans when necessary, and only then executes it. Obstacle-aware contact execution instead selects the contact position so that the contact itself avoids the obstacle. **SAFEHARNESS** attains 71.9% task success and 87.5% collision avoidance, surpassing the previous SOTA by 6.5% and 27.0%, respectively. Compared with the same agent without harnesses, these results represent $2.3\times$ and $1.5\times$ improvements, respectively.

1 INTRODUCTION

Coding agents have emerged as a promising paradigm for robot manipulation. Language models can write robot controllers as programs. Code as Policies [27] set the pattern: the model composes perception and control APIs into a program, and the program is the policy. Later agents rewrite their own controller code after a failure [24], keep a library of skills that worked [32], and spend more test-time compute for more reliability [13]. Frontier agents now drive real robots without robot-specific training [21, 45]. Since the release of GPT-6 Astra [34], many groups have shown promising results with it, and a first controlled study has measured them [38]. With no robot-specific training, Astra completes a zero-shot pick-and-place suite on a Franka arm almost perfectly. Paired with a frozen $\pi_{0.5}$, it extends to bimanual, contact-rich tasks. Astra has no robot policy inside it. Every robot result attributed to it belongs to the harness around it, the tools the agent is given to call. Coding agents make this division of labor explicit. Harness VLA [50] lets the agent explore a scene, keep what worked as a skill memory, and reuse it on later tasks, calling a frozen VLA only where the task turns contact-rich. Xu et al. [46] extends it to long horizons.

None of this work has asked whether such paradigm on robot manipulation is safe. The studies above score a rollout on whether the goal was reached, and the manipulation benchmarks they inherit do the same [28, 10]. What the robot touched on the way is never measured. Yet a collision in a cluttered workspace damages hardware, injures people, and destroys property [14]. Success does not imply safety. Across a model generation the two can move in opposite directions [16, 12]. Scaling up training data does not resolve this either. Training the same policy on ten times as many collision-free

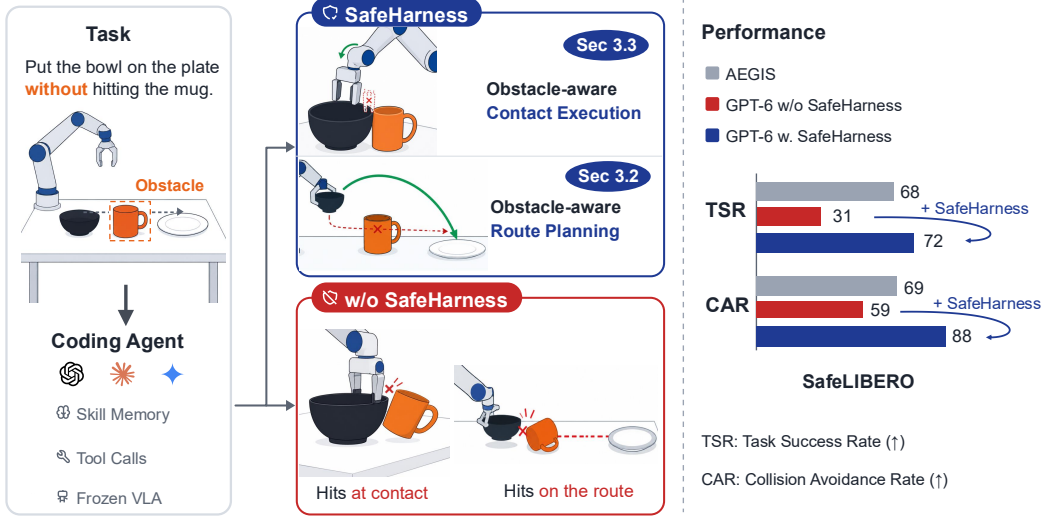


Figure 1: **SAFEHARNES puts safety on par with task completion.** Coding agent receives a task instruction with a safety constraint, along with observation images. **SAFEHARNES** then equips it with obstacle-aware route planning (Sec. 3.2) and obstacle-aware contact execution (Sec. 3.3) to enforce safety throughout the task.

demonstrations lowers its collision rate by under three points [10]. Completing the task and touching nothing else are two halves of one requirement.

To solve this problem, we first evaluate the coding agent on robot safety benchmark. Upon evaluation, *we observe that coding agent performs poorly under a safety constraint.* The coding agent completes the task but collides with the obstacle in most cases. It treats the task goal as the core objective and neglects that safety has to be satisfied as well. The cause is not recognition, since the agent identifies the obstacle correctly. Nor is it instruction. Adding the safety requirement to the prompt does not change the behavior: the agent restates the constraint and then violates it [31, 47]. The reasons follow the decomposition that coding agent itself gives, a route through free space and a contact-rich behavior at its end. First, for the route ①, the model has no notion of a path that clears the obstacle, and it takes the shortest one. Second, it has no notion of replanning when the route it chose turns infeasible midway. Third, for the contact ②, it does not recognize that the contact itself carries a safety constraint. A stronger planner does not close the gap (Sec. 4.3), as also reported at scale [49]. The safety constraint is simply never a core priority.

We propose **SAFEHARNES** to address the gap that the coding agent fails to accord safety the same priority as task completion. **SAFEHARNES** addresses this gap by equipping the model with two obstacle-aware harnesses that give it the ability to prioritize the safety constraint (Fig. 1). *Obstacle-aware route planning* (Sec. 3.2) handles ①. The objects in the scene are grounded as bounding boxes [7], and a route over them is drawn as a sequence of waypoints. The agent plans the route ahead, verifies it, replans when a problem is met along the way, and finally executes the path that has been cleared. The safety constraint is thereby held at the same top priority as the task goal. *Obstacle-aware contact execution* (Sec. 3.3) handles ②. The contact strategy, meaning the exact contact position and whether the gripper rotates, is decided by whether an obstacle stands next to the contact location. The same question is asked where the object is picked and where it is placed.

On SafeLIBERO [15] benchmark, **SAFEHARNES** with GPT-6 reaches 71.9% task success and 87.5% collision avoidance, surpassing the previous SOTA by 6.5% and 27.0%, respectively. Compared with the same agent without harnesses, these results represent $2.3\times$ and $1.5\times$ improvements, respectively. It confirms that the safety improvement comes from our harness rather than from the model (see Fig. 1 right).

Our core contributions are:

- **Coding agents for safe robot control.** We study coding agent for robot control under a requirement it has never been scored on: finish the task under safety constraint. To our knowledge, this is the first coding agent for robot manipulation whose harness itself enforces safety: obstacle avoidance is guaranteed throughout the task, including its contact-rich phases. It is also the first such agent evaluated on a collision-scored manipulation benchmark.

- **SAFEHARNESS.** Our two obstacle-aware harnesses treat safety as a first-class objective, on equal footing with task completion. ❶ Route planning plans, verifies, guards, and replans every free-space route. ❷ Contact execution selects the contact position and wrist rotation according to the spatial relation between the target and its neighboring obstacle.
- **Results.** **SAFEHARNESS** attains 71.9% task success and 87.5% collision avoidance on Safe-LIBERO, surpassing the previous SOTA by 6.5% and 27.0%, respectively. Compared with the same agent without harnesses, these results represent $2.3\times$ and $1.5\times$ improvements, respectively.

2 RELATED WORK

From VLAs and world-action models to coding agents. Vision-language-action (VLA) models map pixels and instructions directly to actions. RT-2 casts action prediction as next-token generation over a vision-language backbone [5]. Octo trains a generalist transformer policy across many robot embodiments [40]. OpenVLA and its optimized fine-tuning recipe scale an open vision-language backbone to new manipulation tasks with modest data [22, 23]. The π_0 and $\pi_{0.5}$ family adds a flow-matching action head for dexterous, contact-rich control [4, 19]. A parallel line pretrains on video before it predicts action. UniPi generates a goal video from language and extracts actions with an inverse-dynamics model [11]. GR-1 and GR-2 pretrain a transformer on internet-scale video and fine-tune it to jointly predict future frames and actions [43, 9], and DreamGen turns a video-generation model fine-tuned on the target embodiment into a source of synthetic action data [20]. WorldVLA and V-JEPA 2-AC couple world-model prediction and action generation inside one architecture [8, 3]. Both families are trained policies, so their competence reaches only as far as their demonstrations, and a new task or embodiment requires robot-specific data or fine-tuning. A direct comparison finds that a world-action model inherits much of the brittleness of the VLA it is built from [51], and in both cases the decision resolved inside an action head resists inspection and correction. A coding agent instead has a language model write the controller as a program that calls perception and control skills, so the policy is a program rather than a network. Written rather than trained, such a policy transfers across tasks without robot-specific data, can be read and edited before execution, and may compose existing skills, learned policies included. The manipulation work that develops this paradigm is surveyed next.

Coding agents in robot manipulation. A coding agent acts by writing and executing code rather than emitting actions or language directly [42, 41]. Code as Policies brought this idea to robot manipulation: a language model composes perception and control APIs into an executable program, and that program, rather than a network’s action head, is the policy [27]. Follow-on work structured what the program could express: ProgPrompt and Instruct2Act structure the prompt and its perception calls [37, 17], RoboCodeX decomposes an instruction into object-centric code with affordance and safety constraints [33], and VoxPoser has the model emit a 3D value map for a motion optimizer to minimize [18]. A second line lets the agent improve its own code: it rewrites controller code from an observed failure [24], repairs its code and grows a skill library through evolutionary search [30], accumulates skills with a human in the loop [32], self-improves on real hardware [45], and trades test-time compute for reliability [13]. The release of GPT-6 Astra shifted what the agent itself can carry [34]. In the first controlled study, Astra completes a zero-shot pick-and-place suite on a Franka arm almost perfectly with no robot-specific training, and, paired with a frozen $\pi_{0.5}$, extends to bimanual, contact-rich tasks [38]. Yet Astra contains no robot policy. Every robot result attributed to it belongs to the harness around it: the tools the agent is given to call. Coding agents make this division of labor explicit. A third line exposes a frozen learned policy as one callable primitive: Harness VLA lets the agent identify the scene once, cross free space with analytic inverse-kinematics primitives, and hand over to the frozen policy only where the task turns contact-rich [50], and BATON extends this design to long-horizon tasks via subtask exploration and transition-aware memory [46]. Across all three lines, the code plans and sequences skills, but a skill executes as a straight-line motion or a learned primitive regardless of what stands in its path. We are the first work to study coding agents for robot manipulation with safety placed on equal footing with task success, whereas prior work pursues task success alone.

Safe control in robotics. Sampling- and optimization-based motion planners construct an explicit route through free space and certify it against a collision model: RRT grows a tree of feasible configurations toward the goal [25], and CHOMP and TrajOpt optimize a trajectory that penalizes

a swept volume against obstacles [35, 36]. These planners fix a route before execution and do not revisit it once tracking begins. Runtime safety filters address that limitation: control barrier functions wrap a controller in a filter that minimally edits each commanded action to keep the system inside a safe set [1], an idea extended to barriers over semantic, scene-grounded regions [6]. Such a filter edits whatever action it is handed; it cannot choose a different grasp side or route. Safety for learned VLA policies is now an active benchmark question. SafeLIBERO adds one obstacle, placed near the target or on the transport path, to four tasks in each of the LIBERO-Spatial, Goal, Object, and Long suites [28], and scores an episode on task success and on whether the obstacle ever moves; it pairs the benchmark with AEGIS, a barrier-function layer over a frozen policy [15]. LIBERO-Safety applies the same measurement to semantic constraints and finds that scaling demonstrations tenfold barely lowers the collision rate [10], and SafeDojo shows that a lower collision rate under reinforcement learning does not carry over to a stricter metric requiring the task to be completed [39]. Across this line, safety is scored or filtered after a policy commits to a motion, rather than built into the decision that produces it. We are the first work to introduce the coding-agent paradigm to safe control in robot manipulation, in which safety enters the decision that produces a motion rather than a filter applied after it, and is supplied by a training-free harness around an unmodified policy.

3 METHOD

3.1 PRELIMINARY

Coding agents have become a strong recipe for robot manipulation: a language model writes the controller as a program, and the program generalizes across tasks without robot-specific training. Safe manipulation has so far been pursued along other routes: planners that certify a trajectory in advance [25, 35, 36], filters that edit each commanded action [1, 6, 15], policies trained under a safety objective [48, 39]. Whether a coding agent can deliver safe manipulation has not been asked, and we are the first to explore it.

We ask it here, and the recipe itself suggests how to approach it. A coding agent composes a task from callable skills and motion primitives, and the natural place to cut such a program is a contact event. Each phase is then a *route* through free space followed by a *contact-rich behavior* at its end. An obstacle can be touched in either part. The two parts nonetheless put it at risk in different ways. A route sweeps a long path across the workspace, whereas a contact-rich behavior sweeps a small volume that cannot be moved away from the target it acts on. We therefore split the safety constraint in the same place. The route constraint asks for a clear path where no single point would collide with the obstacle. The contact constraint asks that the contact-rich behavior be executed strategically when an obstacle is nearby. Neither constraint implies the other, so an episode is safe only when both of them hold. The split gives **SAFEHARNES**S its two components (Fig. 2). *Obstacle-aware route planning* (Sec. 3.2) is responsible for the route constraint ❶. *Obstacle-aware contact execution* (Sec. 3.3) is responsible for the contact constraint ❷. Both components run once per phase, so a task with two sub-goals repeats them twice.

3.2 OBSTACLE-AWARE ROUTE PLANNING

A route is decided before the arm moves. Planning and execution are asymmetric in cost: a plan can be revised freely, whereas a motion already issued acts in the workspace and cannot be recalled, so errors are best caught while they are still words rather than movements. **SAFEHARNES**S accordingly plans the route of a phase ahead, verifies the plan against the obstacle, and executes only a plan that has passed verification.

Route planning begins anew with each phase rather than once for the whole task. Once the object is in the gripper, arm and object move as one body, and the clearance a route requires changes with it. A route cleared for the empty gripper is not cleared for the loaded one. The first step of a phase is visual grounding. The target and the obstacle are grounded as bounding boxes by a segmentation model. On that view the model is asked for a route as a polyline of waypoints, under two conditions: it must end at the target, and it must not cross the obstacle box.

The proposal obtained this way is not yet a route. Before any motion is issued, it is checked against the obstacle box outside the language model; the check covers the gripper and, when an object is

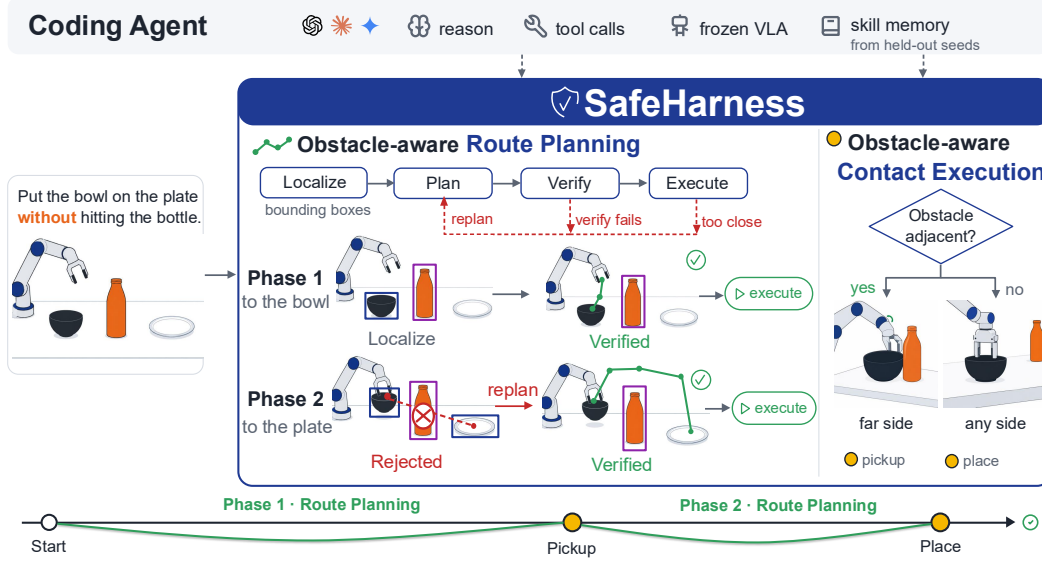


Figure 2: **SAFEHARNESS overview.** The task is cut at its contact events into phases, and both components run once per phase. ❶ *Obstacle-aware route planning*: the scene is grounded as boxes, the route of the current phase is planned, verified against the obstacle, and only then executed, with a failed verification or a halt during execution sending the loop back to planning. ❷ *Obstacle-aware contact execution*: the contact target is tested for an adjacent obstacle, and the contact is made from the far side whenever one is found.

held, the object as well. A candidate that intrudes into the box is rejected and the model replans, until a route passes and is released for execution.

A route released for execution carries, however, only a limited guarantee. The verifier evaluates a single condition, namely whether the route intersects the obstacle, and it does so on an approximately measured scene. Other physical constraints that arise during execution fall outside its scope, and a route that passes verification may still fail against them. Replanning is therefore triggered at two points. The first occurs at verification, before any motion is issued: if the proposed route would enter the obstacle box, it is rejected and a new route is planned. The second occurs during execution, when the robot encounters conditions the verifier did not evaluate. The arm may approach the obstacle more closely than the plan predicted, or it may become stuck partway along the route due to a physical constraint that the bounding boxes did not capture. In either case, execution halts and a new route is planned from the pose at which the motion stopped. Planning can thus be invoked repeatedly within a phase rather than performed once at its outset.

Beyond replanning, execution imposes one further discipline: economy of observation. The natural impulse is to consult the route-plan image at every step, yet each consultation appends the same view to the context window, and a context window inflated by repeated identical observations degrades the quality of the model’s subsequent predictions. **SAFEHARNESS** therefore bounds the number of times the route-plan image may be read during the execution of a route. The bound removes redundant views without withholding information, since a repeated image lengthens the context window but adds nothing to what the model knows.

Taken together, the phase runs as a plan–verify–execute cycle: a route is proposed on the grounded scene, admitted only after it clears the obstacle box, and executed under the same check, with planning re-entered whenever verification or execution rejects the current route. What the cycle delivers is a path that has been confirmed clear before and during motion, ending just above the contact target at a pre-grasp or a release pose, where the phase is handed to the contact step (Sec. 3.3). A long-horizon task is covered by repeating this cycle phase by phase rather than by a longer plan.

3.3 OBSTACLE-AWARE CONTACT EXECUTION

The second harness addresses the contact motion, the segment of the phase that the route does not cover. The route terminates above the contact target, at a pre-grasp or a release pose; below this pose

the gripper opens, descends, and closes, and this motion sweeps a volume that route verification does not examine. Moreover, whereas the route is free to bend around the obstacle, the contact motion is anchored to the target and cannot avoid its vicinity. When the obstacle is adjacent to the target, collisions therefore occur in this final segment even when the route itself is collision-free, and the contact motion must be planned with respect to the obstacle as well.

The contact step is resolved in the same manner as the route: the strategy is determined anew at each contact rather than fixed once for the entire task. Each contact begins with an adjacency test between the obstacle and the contact target. If no obstacle is adjacent, any contact strategy is admissible and the step proceeds as it would in an uncluttered scene. If an obstacle is adjacent, the contact is constrained to approach from a direction that maintains clearance between the gripper and the obstacle, and among the directions that satisfy this condition, the direction farthest from the obstacle is selected. If no direction satisfies the condition, the gripper is reoriented so that its opening axis runs tangent to the obstacle, and the payload is lowered vertically in order to avoid lateral motion toward the obstacle. The selection of the contact strategy is therefore governed by obstacle clearance at every contact.

3.4 DISCUSSION: LONG-HORIZON CODING AGENT

Beyond describing what **SAFEHARNES**S adds, we return to the question from which this work began: why a coding agent that manipulates competently ceases to be safe once a constraint is introduced. The agent states the constraint at the outset, restates it in its plan, and nevertheless fails to act on it. We attribute this behavior to the long-horizon nature of the coding-agent setting. An episode constitutes a single conversation spanning hundreds of steps, in which each tool call appends its result and each observation appends an image. By the time the arm approaches the obstacle, the context has accumulated a long record of tool calls and images and extends to tens of thousands of tokens, while the single sentence that named the obstacle remains near its beginning. Language models use such a sentence less reliably as it recedes: recall degrades for content in the middle of a long context [29], instruction following deteriorates with context length [44], adherence to the system prompt drifts within a few turns [26], and even trained safety refusals weaken under a sufficiently long context [2]. Our rollouts exhibit the embodied form of this effect: the constraint appears in the early reasoning steps, disappears from subsequent ones, and the plan thereafter optimizes the goal alone (Sec. 4.4). A stronger planner extends the horizon over which the sentence survives but does not alter the mechanism (Sec. 4.3).

The design of **SAFEHARNES**S follows from this observation. A rule that must hold at step three hundred cannot rely on a sentence written at step zero; it must instead be re-applied at that step by a mechanism external to the context. The verification tools of **SAFEHARNES**S serve this role: they are stateless with respect to the conversation, impose no burden on the planner’s memory, and return a result the planner can act on in the following step. For an invariant that must persist across a long episode, the tool interface is therefore a more reliable carrier than the prompt, and we expect this conclusion to extend to workspace limits, force limits, and other constraints of the same kind.

4 EXPERIMENTS

Benchmark. SafeLIBERO [15] takes four representative tasks from each of the LIBERO-Spatial, Goal, Object, and Long suites [28] and adds to each one obstacle object: a moka pot, storage box, milk carton, wine bottle, mug, or book. The task language never mentions the obstacle. Each task appears once per level. In Level I the obstacle stands close to the target object, so it interferes with grasping and releasing (②). In Level II it stands away from the target but on the natural transport path, so it interferes with the route (①). We evaluate **SAFEHARNES**S on SafeLIBERO with 10 seeds/task for all 32 tasks. We reuse the coding-agent loop, the frozen $\pi_{0.5}$ checkpoint [19], and the original Harness VLA skill memory for LIBERO [28], all unchanged, and adopt the same segmentation model, SAM-3 [7]. The only difference is that we equip the agent with the two obstacle-aware harnesses of Sec. 3. Throughout Tab. 1 and Tab. 2, the coding-agent backbone is GPT-5.5 or GPT-6 [34].

Table 1: SafeLIBERO [15] : per-suite and average task success rate (TSR) and collision-avoidance rate (CAR), in %. **SAFEHARNESS** instantiated with two coding-agent backbones (GPT-5.5 and GPT-6-astra). Best per column in bold.

Method	Spatial		Goal		Object		Long		Average	
	TSR	CAR	TSR	CAR	TSR	CAR	TSR	CAR	TSR	CAR
OpenVLA-OFT [23]	36.5	8.3	24.3	19.5	28.8	11.5	15.3	6.0	26.2	5.7
$\pi_{0.5}$ [19]	59.3	14.0	60.0	20.0	57.5	18.0	54.3	16.5	57.8	17.1
AEGIS [15]	68.5	68.0	82.8	76.5	72.5	71.3	46.3	59.8	67.5	68.9
SAFEHARNESS (GPT-5.5)	72.0	75.0	52.0	88.0	82.0	85.0	72.0	95.0	70.0	86.0
SAFEHARNESS (GPT-6)	75.0	75.0	75.0	100.0	75.0	75.0	62.5	100.0	71.9	87.5

4.1 EXPERIMENTAL SETUP

Metrics. An episode *succeeds* when the goal predicate becomes true, and it is *safe* when the obstacle is never displaced. Displacement is scored as in the official SafeLIBERO evaluator: an episode is collided if the obstacle moves by more than 1 mm at any point. We report two metrics: task success rate (TSR), the percentage of episodes in which the task goal is achieved, and collision-avoidance rate (CAR), the percentage of episodes in which no obstacle is displaced.

Models. **SAFEHARNESS** baseline is built on Harness VLA [50]. We reuse its coding-agent loop, its frozen $\pi_{0.5}$ [19] checkpoint, and the original Harness VLA skill memory for LIBERO [28], all unchanged. We also adapt the same segmentation model of SAM-3[7]. Our main difference is equipping with the two obstacle-aware harnesses of Sec. 3. The coding agent backbone is GPT-5.5 or GPT-6-astra [34], both in Tab. 1 and in Tab. 2. We compare our methods against two foundation VLA models - $\pi_{0.5}$ [19] and OpenVLA-OFT[23], , along with previous SOTA method - AEGIS [15] ($\pi_{0.5}$ with the barrier-function safety layer).

4.2 MAIN RESULTS

SAFEHARNESS attains the best average TSR and the best average CAR in Tab. 1, at 71.9 and 87.5 under GPT-6, ahead of AEGIS [15], the strongest prior method, on both axes. The baselines separate the two axes, whereas our rows do not. The frozen policies score respectably on success and poorly on safety, with $\pi_{0.5}$ [19] reaching 57.8 TSR against 17.1 CAR. The barrier layer of AEGIS closes most of that distance on safety, to 68.9 CAR, while its success stays close to the frozen policy it wraps. **SAFEHARNESS** raises both columns at once, and the two planners land within two points of each other on either metric.

The pattern of the baselines follows from where each mechanism acts. A frozen policy never had the obstacle in its training objective, so nothing inside it distinguishes an approach that clears the obstacle from one that sweeps through it. A barrier layer sits downstream of the policy and edits the commanded action, which lets it veto a motion that violates the constraint but gives it no means of proposing a better one. Vetoing is enough to keep most episodes clear, yet what it removes is also the motion the policy needed, so the cost appears wherever the filter has to intervene repeatedly. On Long, whose two sub-goals call for two such interventions, AEGIS ends below the frozen policy it wraps on task success.

SAFEHARNESS acts earlier, while the route and the contact side are still being chosen, so the constraint enters the decision rather than the correction. A route is committed only after it has been checked against the obstacle, and a contact is made from a side selected with the obstacle in view. The motion finally issued is therefore admissible at the moment it is issued, and nothing has to be subtracted from it afterwards. Success and safety consequently rise together instead of trading against each other, which is what separates the harnesses rows from the filtered baseline. In the cells where a baseline completes more tasks, our collision-avoidance rate remains the higher of the two, so the episodes we give up end without contact, which is the preferable way to fail. **SAFEHARNESS** is nonetheless not collision-free, and it still displaces the obstacle in about one episode in eight.

Table 2: Ablation on model family and the source of safety gains on SafeLIBERO [15]. TSR/ CAR in %. **Model**: the language model drives motion and primitives directly. **Model + Skills**: adds skill memory and the frozen VLA. **SAFEHARNES**: further adds the obstacle-aware harnesses of Sec. 3.

Planner	Agent	Level I		Level II		All 32	
		TSR	CAR	TSR	CAR	TSR	CAR
GPT-5.5	Model + Skills	12.5	18.8	43.8	43.8	28.0	31.0
	SAFEHARNES	76.0	86.0	64.0	85.0	70.0	86.0
GPT-6	Model	12.5	50.0	0.0	50.0	6.0	50.0
	Model + Skills	31.0	69.0	31.0	50.0	31.0	59.0
	SAFEHARNES	81.2	87.5	62.5	87.5	71.9	87.5

4.3 ABLATION: WHERE THE CODING AGENT FAILS, AND WHAT FIXES IT

Tab. 2 fixes the 32 scenes and the inputs and varies only what the planner is given, crossing two planners with three agents and reporting every cell by level. Its skills-only rows are code as policy under a constraint that arrives the way a coding agent normally receives one, as language in the task description. That channel is known to secure agreement rather than behavior [31, 47], and the agent carrying it here already solves the obstacle-free form of these scenes [28]. **SAFEHARNES** differs from that agent by the two obstacle-aware harnesses and by nothing else, so the vertical distance measures the harnesses while the horizontal distance measures model scale. Read by level, the table locates the failure; read across the planners, it says whether a better model would repair it.

The collision-avoidance rate of the model-only agent cannot be read as evidence of safe behavior. That agent reaches 6.0 TSR and 50.0 CAR, and the second figure is vacuous, because an arm that fails before it arrives displaces nothing. Its failures are those of geometric reasoning without skills: it pinches bowls and mugs from above until the grip slips, and it re-inspects the scene only once the episode has already gone wrong. Restoring the skill memory and the frozen VLA restores manipulation, raising task success fivefold under GPT-6, while collision avoidance moves by nine points. Under the weaker planner the same agent keeps barely three episodes in ten clear of the obstacle, a lower rate than the unskilled agent achieved by failing early. Competence therefore does not become safety, and it is competence that makes the collisions this benchmark scores possible at all.

Model scale improves the decision that can be made locally and leaves the one that cannot. Across the two skills-only rows the stronger planner gains a few points of task success and a great deal of collision avoidance. Almost all of that safety gain sits at Level I, where CAR rises from 18.8 to 69.0. At Level II the same step forward buys six points of safety and gives back nearly thirteen points of success. The asymmetry follows the two halves of a phase (Sec. 3.1). Which side of a target to approach is a local decision, legible in a single view, and a stronger planner decides it better. A route is not local, since it has to hold over a long motion, and a planner with no way to examine a route before it is flown has nothing to improve upon. An audit of language models reports the same shape at scale, with planning ability growing and safety awareness staying flat [49].

The harnesses raise both metrics under either planner and remove the dependence of the failure profile on the planner. Adding them raises task success by more than forty points and lifts collision avoidance to eighty-five or above at both levels, under either planner, with no change to the prompt, the memory, or the VLA. What changes is not only the height of the two metrics but also the shape of the failure. The level-wise spread in collision avoidance that distinguished the two planners closes, so each harnessed agent is about as safe at Level I as at Level II. Where the agent remains unsafe thus stops depending on which model is planning. The gap between GPT-6 [34] with and without the harnesses is an order of magnitude wider than the gap between the two planners once both are harnessed. Once the harness verifies the route and fixes the contact side, the planner chooses only among admissible options, which leaves a stronger planner little to add. Safety here is therefore a property of the harness rather than of the model it wraps. What the harnesses do not equalize is task success across levels, which stays lower at Level II under both planners, 64.0 against 76.0 and 62.5 against 81.2. Because collision avoidance is flat across the levels, the residual cost falls on completion rather than on safety, most plausibly because a verified detour lengthens an episode that runs under a fixed step budget.

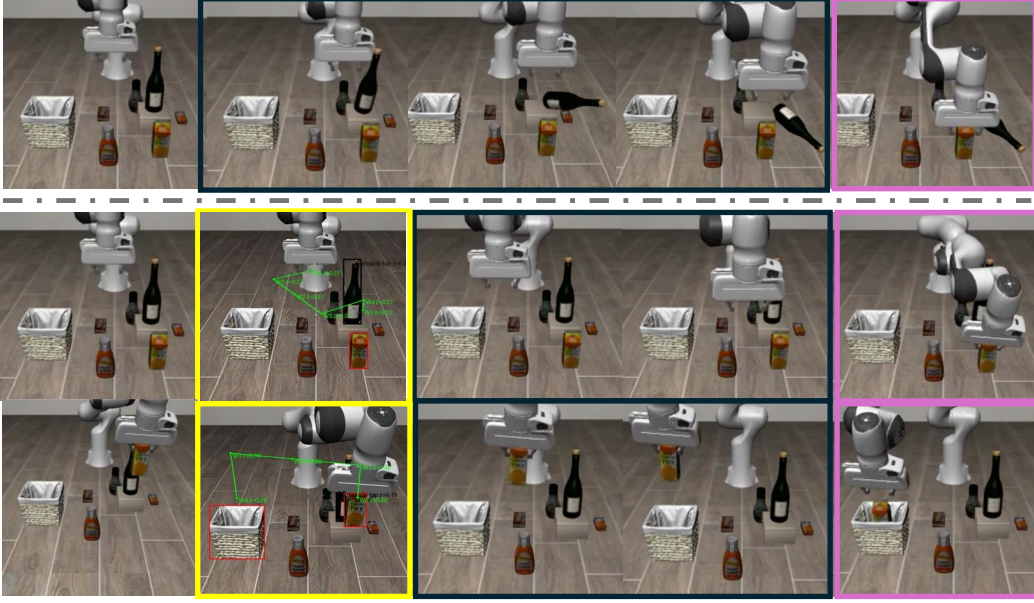


Figure 3: **Successful SAFEHARNESS rollout vs. failure without it on SafeLIBERO.** The task is to place the orange juice in the basket without hitting the wine bottle. Without **SAFEHARNESS** (top row), hits the obstacle en route to the grasp; with **SAFEHARNESS** (bottom rows), the agent detours around it and finishes collision-free. Panel frames: yellow = route planning (bounding boxes and waypoints), black = verified-route execution, purple = contact-rich moments.

4.4 QUALITATIVE ANALYSIS

Fig. 3 contrasts a failed and a successful rollout on the same SafeLIBERO [15] scene. The task instructs the agent to pick up the orange juice and place it in the basket while not colliding with the wine bottle. The skills-only agent (top row) acknowledges the bottle in its reasoning, and the acknowledgment does not survive into the motion. Its traces show a consistent pattern: the constraint is mentioned in the first few reasoning steps and then drops out, and the plan that follows takes the shortest line at grasp height, straight through the bottle. The agent optimizes for the task goal alone, as if success were defined without the constraint rather than under it. **SAFEHARNESS** (bottom rows) instead clears the bottle and completes the task. The constraint holds because every route is checked before the arm moves, rather than considered once and forgotten. In Long this loop runs twice per episode, once per sub-goal, and the second leg re-uses the obstacle box measured for the first.

We now walk through the successful rollout phase by phase, one per sub-goal, each running the full loop of Sec. 3.2 and Sec. 3.3. In the first phase the agent grounds the orange juice and the wine bottle as boxes and plans the route to the pick. The bottle blocks the way, so the route detours around it, passes verification, and the executed motion follows the verified plan. In the second phase the agent transports the juice to the basket. With the bottle no longer in the way, the route proceeds directly to the basket and is verified as proposed. The two phases together show that the harness plans a detour only when an obstacle demands one and otherwise takes an efficient route, so safety is not bought by timidity. At both contact moments, the grasp of the juice and the release into the basket, the adjacency test finds no obstacle beside the target, so contact execution admits the default strategy and no side selection is triggered. The rollout thus exercises every decision point of **SAFEHARNESS**, and each component intervenes exactly where the scene requires it.

5 CONCLUSION

Safe manipulation around an obstacle has been treated as a problem of a better policy or a better filter around it; we have argued it is, to a first approximation, a problem of *where the obstacle is relative to the task*. When the obstacle is on the route, the free-space path must be planned and checked

against the obstacle boxes, and execution must be held to the checked route. When the obstacle is next to the target, the route is not the problem; the grasp side, the wrist orientation, and the placement approach are. **SAFEHARNESS** moves both halves into the harness a coding agent runs on, so that the planner chooses among admissible options rather than being asked to remember rules, and it does so on a frozen VLA, with no gradient anywhere. On SafeLIBERO the combination reaches 71.9% task success and 87.5% collision avoidance, 40.9 and 28.5 points above the same agent without **SAFEHARNESS**, which relies solely on the safety requirement in the task description. Instructions to a language-model planner are not constraints; constraints have to live where the robot moves.

Limitations. Our evaluation follows the protocol of SafeLIBERO benchmark, which declares the obstacle categories and places a single obstacle per scene, and our obstacle model matches this setting with one axis-aligned box; scenes with several obstacles would need a richer planner behind the same interface, which the interface is designed to admit. The contact-side test is tuned for a parallel-jaw gripper, the standard end-effector of the benchmark and of the base harness we build on. Grounding quality is inherited from the segmentation model, as in all agents of this line, and a grounding failure surfaces as a rejected route rather than a collision, since verification runs on the grounded boxes. Each episode costs minutes of planner time, a cost shared by test-time-compute agents in general and amortizable by caching verified routes as skills. Like the base harness and the benchmark, our evaluation is simulation-first.

REFERENCES

- [1] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *European Control Conference (ECC)*, 2019.
- [2] Cem Anil, Esin Durmus, Nina Panickssery, Mrinank Sharma, Joe Benton, Sandipan Kundu, Joshua Batson, Meg Tong, Jesse Mu, Daniel Ford, Francesco Mosconi, Rajashree Agrawal, Rylan Schaeffer, Naomi Bashkansky, Samuel Svenningsen, Mike Lambert, Ansh Radhakrishnan, Carson Denison, Evan J. Hubinger, Yuntao Bai, Trenton Bricken, Timothy Maxwell, Nicholas Schiefer, James Sully, Alex Tamkin, Tamera Lanham, Karina Nguyen, Tomasz Korbak, Jared Kaplan, Deep Ganguli, Samuel R. Bowman, Ethan Perez, Roger Grosse, and David Duvenaud. Many-shot jailbreaking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. doi: 10.52202/079017-4121.
- [3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [5] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [6] Lukas Brunke, Yanni Zhang, Ralf Römer, Jack Naimier, Nikola Staykov, Siqi Zhou, and Angela P Schoellig. Semantically safe robot manipulation: From semantic scene understanding to motion safeguards. *IEEE Robotics and Automation Letters*, 2025.
- [7] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris Coll-Vinent, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, et al. SAM 3: Segment anything with concepts. In *International conference on learning representations*, 2026.
- [8] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, et al. Worldvla: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.

- [9] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, et al. GR-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024.
- [10] Rongxu Cui, Zongzheng Zhang, Jingrui Pang, Haohan Chi, Jinbang Guo, Saining Zhang, Shaoxuan Xie, Xin Jin, Yao Mu, Jiaolong Yang, et al. Libero-safety: A comprehensive benchmark for physical and semantic safety in vision-language-action models. In *European Conference on Computer Vision*. Springer, 2026.
- [11] Yilun Du, Mengjiao Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Joshua B. Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [12] Jialiang Fan, Weizhe Xu, Oleg Sokolsky, Insup Lee, and Fanxin Kong. Safevla-bench: A benchmark for the success-safety gap in vision-language-action models. *arXiv preprint arXiv:2606.00773*, 2026.
- [13] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Dantong Niu, Fei-Fei Li, et al. Cap-x: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026.
- [14] Sami Haddadin, Alessandro De Luca, and Alin Albu-Schäffer. Robot collisions: A survey on detection, isolation, and identification. *IEEE Transactions on Robotics*, 2017.
- [15] Songqiao Hu, Zeyi Liu, Shuang Liu, Jun Cen, Zihan Meng, Shihefeng Wang, Xiang Li, and Xiao He. Vlsa: Vision-language-action models with plug-and-play safety constraint layer. *arXiv preprint arXiv:2512.11891*, 2025.
- [16] Chengyue Huang, Khang Vo Huynh, Sebastian Elbaum, Zsolt Kira, and Lu Feng. Safemanip: A property-driven benchmark for temporal safety evaluation in robotic manipulation. *arXiv preprint arXiv:2605.12386*, 2026.
- [17] Siyuan Huang, Zhengkai Jiang, Hao Dong, Yu Qiao, Peng Gao, and Hongsheng Li. Instruct2act: Mapping multi-modality instructions to robotic actions with large language model. *arXiv preprint arXiv:2305.11176*, 2023.
- [18] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023.
- [19] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [20] Joel Jang, Seonghyeon Ye, Zongyu Lin, Jiannan Xiang, Johan Bjorck, Yu Fang, Fengyuan Hu, Spencer Huang, Kaushil Kundalia, Yen-Chen Lin, et al. Dreamgen: Unlocking generalization in robot learning through video world models. *arXiv preprint arXiv:2505.12705*, 2025.
- [21] Mengzhao Jia, Yang Lin, Xixin Zhang, Zhihan Zhang, Xiaobai Liu, and Meng Jiang. Agent as policy for robotic manipulation. *arXiv preprint arXiv:2609.12541*, 2026.
- [22] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [23] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [24] Vaishak Kumar. Act-observe-rewrite: Multimodal coding agents as in-context policy learners for robot manipulation. *arXiv preprint arXiv:2603.04466*, 2026.
- [25] Steven M. LaValle. Rapidly-exploring random trees: A new tool for path planning. *Research Report 9811*, 1998.

- [26] Kenneth Li, Tianle Liu, Naomi Bashkansky, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Measuring and controlling instruction (in)stability in language model dialogs. In *Conference on Language Modeling (COLM)*, 2024. arXiv:2402.10962.
- [27] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. arXiv:2209.07753.
- [28] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, 2023.
- [29] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl.a_00638. arXiv:2307.03172.
- [30] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzen Xu, Guanya Shi, Ken Goldberg, Ang Chen, et al. Aspire: Agentic skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.
- [31] Xiaoya Lu, Zeren Chen, Xuhao Hu, Yijin Zhou, Weichen Zhang, Dongrui Liu, Lu Sheng, and Jing Shao. IS-Bench: Evaluating interactive safety of VLM-driven embodied agents in daily household tasks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [32] Yuan Meng, Zhenguo Sun, Max Fest, Xukun Li, Zhenshan Bing, and Alois Knoll. Growing with your embodied agent: A human-in-the-loop lifelong code generation framework for long-horizon manipulation skills. *arXiv preprint arXiv:2509.18597*, 2025.
- [33] Yao Mu, Junting Chen, Qinglong Zhang, Shoufa Chen, Qiaojun Yu, Chongjian Ge, Runjian Chen, Zhixuan Liang, Mengkang Hu, Chaofan Tao, et al. Robocodex: Multimodal code generation for robotic behavior synthesis. *arXiv preprint arXiv:2402.16117*, 2024.
- [34] OpenAI. GPT-6 Astra: A new generation of intelligence, 2026. Accessed 2026-09-14.
- [35] Nathan Ratliff, Matthew Zucker, J. Andrew Bagnell, and Siddhartha Srinivasa. Chomp: Gradient optimization techniques for efficient motion planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2009.
- [36] John Schulman, Yan Duan, Jonathan Ho, Alex Lee, Ibrahim Awwal, Henry Bradlow, Jia Pan, Sachin Patil, Ken Goldberg, and Pieter Abbeel. Motion planning with sequential convex optimization and convex collision checking. *The International Journal of Robotics Research*, 2014.
- [37] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*, 2023.
- [38] Jiayi Su, Yixin Zheng, Mi Yan, Yizhou Zhou, Li Yi, Zhizheng Zhang, and He Wang. GPT 6 Astra as an embodied policy: A comparative study of direct end-effector control and hybrid control with $\pi_{0.5}$, 2026.
- [39] Kai Tang, Peidong Jia, Zhong Chu, Jixian Wu, Rui Ma, Jiajun Cao, Fangyuan Zhao, Sixiang Chen, Yichen Guo, Xiaowei Chi, et al. Safedojo: Safe reinforcement learning for vla via interactive world model. *arXiv preprint arXiv:2606.20698*, 2026.
- [40] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [41] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

- [42] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. *arXiv preprint arXiv:2402.01030*, 2024.
- [43] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, 2024.
- [44] Xiaodong Wu, Minhao Wang, Yichen Liu, Xiaoming Shi, He Yan, Xiangju Lu, Junmin Zhu, and Wei Zhang. Lifbench: Evaluating the instruction following performance and stability of large language models in long-context scenarios. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL, Long Papers)*, 2025. arXiv:2411.07037.
- [45] Wenli Xiao, Jia Xie, Tonghe Zhang, Haotian Lin, Letian Fu, Haoru Xue, Jalen Lu, Yi Yang, Cunxi Dai, Zi Wang, et al. ENPIRE: Agentic robot policy self-improvement in the real world. *arXiv preprint arXiv:2606.19980*, 2026.
- [46] Bingxin Xu, Yuzhang Shang, and Emilio Ferrara. Don’t drop the baton: Long-horizon robot manipulation via agentic subtask exploration and transition-aware memory. *arXiv preprint arXiv:2608.16889*, 2026.
- [47] Sheng Yin, Xianghe Pang, Yuanzhuo Ding, Menglan Chen, Yutong Bi, Yichen Xiong, Wenhao Huang, Zhen Xiang, Jing Shao, and Siheng Chen. Safeagentbench: A benchmark for safe task planning of embodied llm agents. *arXiv preprint arXiv:2412.13178*, 2024.
- [48] Borong Zhang, Yuhao Zhang, Jiaming Ji, Yingshan Lei, Yishuai Cai, Josef Dai, Yuanpei Chen, and Yaodong Yang. Safevla: Towards safety alignment of vision-language-action model via constrained learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. Spotlight; arXiv:2503.03480.
- [49] Tao Zhang, Kaixian Qu, Zhibin Li, Jiajun Wu, Marco Hutter, Manling Li, and Fan Shi. Using large language models for embodied planning introduces systematic safety risks. *arXiv preprint arXiv:2604.18463*, 2026.
- [50] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaying Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, et al. Harness vla: Steering frozen vlas into reliable manipulation primitives via memory-guided agents. *arXiv preprint arXiv:2607.08448*, 2026.
- [51] Zhanguang Zhang, Zhiyuan Li, Behnam Rahmati, Rui Heng Yang, Yintao Ma, Amir Rasouli, Sajjad Pakdamansavaji, Yangzheng Wu, Lingfeng Zhang, Tongtong Cao, et al. Do world action models generalize better than vlas? a robustness study. *arXiv preprint arXiv:2603.22078*, 2026.